

A Contract-Oriented Review of StradFixed2.wl

Correctness boundaries, search-state defects,
resource contracts, and a proposed third revision

Prepared for Vladimir Reshetnikov
September 2026

Reviewed commit	9000d6f533a68ee7659a6830ad25bba401f800a5
Reviewed file	src/corrected/StradFixed2.wl
Baseline Git blob	0bdeded347385a55a044fdd35883006d053685dc
Proposed package	StradFixed3.wl, context RadicalDenest3‘
Baseline integrity	41,416 bytes; Git-blob hash verified
Executed checks	396 independent exact-mathematical and structural checks
Native regression suite	40 Wolfram tests supplied; not executed here

Reading the conclusions. The strongest findings concern domain admission, loss of previously accepted improvements, a missed index-reduction opportunity, budget-sensitive negative caching, and resource-contract gaps. This review does *not* demonstrate an incorrect returned numerical value on the intended closed exact-algebraic input class. The proposed package is a reproducible source revision, not a claim of completed native-kernel qualification.

Abstract

The second corrected Strad implementation has a valuable architecture: untrusted candidate generation is separated from an exact-equality acceptance gate, and symbolic hosts are processed through restricted congruence-preserving traversal. The important remaining questions are therefore more specific than whether a particular radical identity happens to simplify. Does the domain validator really establish algebraicity? Can a later unsuccessful search discard a previously certified improvement? Do recursive caches remember a mathematical result or merely an exhausted budget? Are the advertised operation and batch limits applied where the work is performed?

This article audits those questions against the pinned source, the unified-B review, the repository survey, selected primary literature, and current Wolfram documentation. It develops explicit witnesses for the sparse multiquadratic ansatz and for index reduction that is useful without recursive progress. It also proves a small-field character-reconstruction method and derives a general odd-index trace–norm kernel. The accompanying package preserves the existing quadratic, cubic, Honsbeek, Kummer, multiplier, rationalization, and traversal machinery while hardening their interfaces. An exact copy of the baseline, a hash-checking build script, a source diff, independent executable checks, and a native regression suite make the proposal auditable and reproducible.

Contents

1	Scope, evidence, and overall assessment	3
1.1	What was reviewed	3
1.2	Evidence levels	3
1.3	What the second correction gets right	3
2	Architecture and intended contracts	4
2.1	The execution pipeline	4
2.2	Four separate invariants	4
3	Finding register	5
4	The exact-algebraic boundary	5
4.1	Exact is not the same as algebraic	5
4.2	Why this matters even without a demonstrated wrong answer	6
4.3	A conservative repair	6
4.4	Make all three equality statuses honest	6
5	Search-state defects and concrete witnesses	7
5.1	Later no-op stages can erase earlier success	7
5.2	Useful index reduction does not require recursive progress	8
5.3	A failed bounded search is not a reusable negative fact	8
6	Resource contracts, option dispatch, and metric details	9
6.1	An overall timeout does not enforce every local timeout	9
6.2	The discriminant batch bound is misplaced	9
6.3	Budgeted work should include preflight and report metrics	9
6.4	Degree limits are output filters, not complexity certificates	10
6.5	Malformed options should reach the validator	10
6.6	Cost accounting should be direct	10
7	Verification of the retained algebraic kernels	11
7.1	Direct square-root denesting	11
7.2	Indirect denesting with a fourth root	11
7.3	Principal roots on the negative real axis	11
7.4	Odd roots in a quadratic field	12
7.5	The Honsbeek quartic proposal is algebraically sound	13
7.6	Polynomial inverses for rationalization	14

8	The sparse multisurd ansatz: a proved limitation	14
8.1	A counterexample inside a small field	14
8.2	Closing the visible prime basis is still not enough	14
9	A bounded character-reconstruction kernel	15
9.1	Square-class basis without an explicit prime-factor basis	15
9.2	Conjugates and the finite transform	15
9.3	Why this captures the missing coset	16
9.4	An explicit reconstruction table	16
9.5	The cost is exponential in field degree	17
10	The proposed source revision	17
10.1	A standalone file, not an undocumented patch layer	17
10.2	Behavior intentionally changed	17
10.3	New options	18
10.4	Usage	18
11	Validation, reproducibility, and remaining qualification	19
11.1	What actually ran	19
11.2	The native regression suite	19
11.3	What should be measured next in a native environment	19
12	Further improvements, in priority order	20
12.1	Make the trusted core smaller and more explicit	20
12.2	Cache mathematical state separately from search coverage	20
12.3	Separate field discovery from radical presentation	20
12.4	Preserve useful partial work without weakening publication	21
12.5	Optimize a small portfolio rather than a single presentation	21
13	Conclusion	21
A	Archive contents and rebuild procedure	21
B	Source locations for the principal findings	22

1 Scope, evidence, and overall assessment

1.1 What was reviewed

The source under review is the 745-line file `src/corrected/StradFixed2.wl` at the commit printed on the title page. The copy included with this article has the exact Git-blob SHA-1 reported by GitHub. The hash is computed using the Git object convention, namely SHA-1 of `blob <byte-count>\0` followed by the file bytes; it is not merely SHA-1 of the file content. The review is therefore about a reproducible version, not a moving `main` branch. The preceding review is `src/code-review/unified-B/unified_analysis_B.tex`; the broader survey is `docs/report/radical_denesting_unified.tex`.^[1, 2, 3]

The literature reading concentrated on the repository’s corrected editions of Jeffrey–Rich, Borodin–Fagin–Hopcroft–Tompkins, and Landau’s 1989 extended abstract. These sources are especially relevant to restricted practical algorithms, square-root/fourth-root distinctions, partial denesting, and the importance of specifying the base field and allowed roots of unity. The repository explicitly identifies these files as editorial re-typesettings, not facsimiles; this review does not silently attribute the editors’ qualifications to the original authors.^[5, 6, 7, 4]

1.2 Evidence levels

The findings use four distinct kinds of evidence. *Static evidence* is an implication of the actual source: for example, an assignment overwrites an incumbent regardless of its quality. *Mathematical evidence* is an exact identity or field-theoretic obstruction proved here. *Documented semantic evidence* comes from Wolfram’s description of a function. *Native execution* would mean running this precise revision in a Wolfram kernel.

The first three kinds are available. The fourth is not: the Wolfram connector was attempted but failed, and no local native kernel was available. Consequently, none of the 40 supplied Wolfram tests is reported as passed. The 396 executed checks are Python/SymPy exact mathematics, finite loop models, baseline-integrity checks, and lexical/source checks. They are not a Wolfram emulator and do not establish Wolfram evaluation semantics or performance.

This separation matters particularly for two results. The root-domain problem is a defect in the predicate’s implication, not an observed wrong numerical answer. The lost-incumbent problem has a concrete control-flow witness and a mathematically natural partial-denesting family, but the default package was not run to measure which instances discover that partial denesting first.

1.3 What the second correction gets right

The dominant positive feature is the single acceptance gate. A proposed replacement must be an explicit radical expression, fit the size limit, be strictly cheaper than the current incumbent, and be certified equal to the *original target* of the island. This applies to specialized kernels, `Simplify`, `Factor`, `ToRadicals`, and user callbacks. Searching heuristically is entirely compatible with this architecture.^[1], lines 224–255

The implementation also keeps `Root` objects from masquerading as denested radical output, isolates `$Assumptions`, restricts traversal through symbolic hosts, bounds the overall multiplier queue at its admission point, and preserves only completed top-level passes on interruption. The polynomial inverse used for denominator rationalization is algebraically correct. The indirect square-root criterion and the principal phase for negative cube roots are also correct; neither should be undone in another cleanup.^[1], lines 261–279, 301–323, 379–396, 648–690

The appropriate conclusion is not that the package should be discarded. It is that the separation between discovery and verification should be extended to *domain admission*, *incumbent state*,

cache state, and resource accounting.

2 Architecture and intended contracts

2.1 The execution pipeline

Layer	Role in the baseline
Public entry points	Strad , DenestRadicals , DenestCore , and DenestReport resolve per-head options and enter a shared session.
Session	A dynamically scoped configuration, deadline, statistics, trace, certificate sample, memo table, and recursion depth are shared across the call.
Traversal	walk visits supported arithmetic/list hosts; island handles exact algebraic subexpressions and recombination.
Discovery	Shape-specific square/cube kernels, linear factors in radical extensions, index reduction, generic CAS proposals, and multiplier search produce candidate presentations.
Acceptance	accept checks grammar, size, cost, and exact equality. acceptWithPolish proposes several alternative presentations.
Publication	A completed pass is committed only after a whole-number equality check when the entire input is numeric. Symbolic hosts rely on certified local replacements and congruence.

This organization is visible directly in the source and is a substantial improvement over a simplifier that assumes its candidate formulas always choose the right branch.[\[1, 2\]](#)

2.2 Four separate invariants

For an original island E and incumbent B , the desired invariants are:

$$B = E, \tag{1}$$

$$\text{cost}(B) \leq_{\text{lex}} \text{cost}(E), \tag{2}$$

$$\text{every stage returns either } B \text{ or a certified cheaper candidate}, \tag{3}$$

$$\text{an exhausted search is not represented as a proof of nonexistence}. \tag{4}$$

The baseline gate is designed to enforce the first two. The later-stage reset violates the third while preserving equality. Budget-insensitive negative memoization violates the fourth. Conflating these failures with incorrect arithmetic would obscure both the diagnosis and the fix.

The cost is a presentation heuristic, not an algebraic invariant:

$$\text{cost}(E) = (o(E), d(E), r(E), \text{LeafCount}(E), b(E)). \tag{5}$$

Here o counts opaque algebraic representations, d is radical depth, r is radical-node count, and b prices exact coefficients. Equivalent algebraic numbers can have very different costs. There is no contradiction in using canonical algebraic values for equality and noncanonical radical expressions for optimization.

3 Finding register

ID	Priority	Finding and evidence	Revision status
F1	High: contract	Exact-looking Root objects are admitted without establishing algebraicity. Static and documented semantic evidence.	Conservative rational-polynomial root grammar.
F2	High: search state	Negative-radical and Kummer stages can reset a better incumbent to the original target. Static witness; partial-denesting example.	Incumbent passed through both stages.
F3	Medium: coverage	Index reduction is discarded unless recursion changes its subproblem and reduces depth. Exact fourth-root witness.	Offer reduced presentation before recursion.
F4	Medium: cache	Unchanged results are cached without recursion/trial-budget context. Static evidence.	Positive incumbents only; path-local cycle guard.
F5	Medium: resources	Full integer factorizations and some expansion/normalization work bypass the operation wrapper; scans and final costs lie outside the outer region.	Explicit wrappers; budgeted scans and cached costs.
F6	Medium: resources	The nominal 24-proposal discriminant batch can reach 46, although the global admission cap holds. Exact finite loop model.	Check batch limit in the inner loop; configurable cap.
F7	Medium: API semantics	A numerical diagnostic can produce the public status " Different " despite exact-decision wording. Previously acknowledged limitation, not a new discovery.	Numerical evidence is diagnostic only.
F8	Low/medium: API	Malformed non-rule option tails can remain unevaluated instead of reaching the documented Failure path.	Broader tail patterns, central validation retained.
F9	Low: metric	Gaussian-rational bit payloads are underpriced; subtractive opaque-node counting is brittle.	Direct recursive accounting and opaque pruning.
F10	Coverage opportunity	The sparse multisurd basis omits valid square-class cosets; an explicit example proves the kernel's miss. Not a proof of whole-package failure.	Small-rank character reconstruction added.

4 The exact-algebraic boundary

4.1 Exact is not the same as algebraic

The problematic clause is short:

```
opaqueQ[e], TrueQ[Precision[e] == Infinity] && FreeQ[e, _Real]
```

It occurs inside `algebraicFormQ`, where `opaqueQ` includes both `Root` and `AlgebraicNumber`. No polynomial or coefficient-domain check is performed.[1], lines 143–166

Wolfram's `Root` is more general than a canonical algebraic-number wrapper. Its documented representations include roots of general equations, polynomial roots with nonrational numeric coefficients, and parameterized polynomial roots. Exactness of representation does not establish membership in $\overline{\mathbb{Q}}$. [8]

For example, consider an indexed representation of a root r of

$$r^5 + \pi r + 1 = 0. \tag{6}$$

The polynomial is strictly increasing on the real line, so it has a unique real root, and that root is not zero. If r were algebraic, then

$$\pi = -\frac{r^5 + 1}{r} \quad (7)$$

would be algebraic, a contradiction. Thus this is an exact number which is not algebraic. An exact-looking indexed `Root[##^5 + Pi ## + 1 &, 1]` retained by the kernel satisfies the old structural acceptance rule without satisfying its claimed mathematical conclusion. The native regression suite checks rejection of this input; the present review does not claim to have observed its exact internal form on a running kernel.

A parameterized `Root[##^5 + a ## + 1 &, 1]` exposes a second issue: the input need not even denote a closed numeric constant. The existing precision/no-`Real` test cannot serve as a proof that parameters are absent.

4.2 Why this matters even without a demonstrated wrong answer

The exact method in `PossibleZeroQ` has a documented guarantee for *explicit algebraic numbers*. The source chooses that method after `exactQ` succeeds. Therefore the validator is part of the proof boundary, not a cosmetic input filter. A validator that admits a larger domain weakens the justification for the subsequent certification call.[9]

The failure mode should nevertheless be stated narrowly. Unsupported roots may simply remain unchanged, cause a timeout, or make the equality oracle return `"Unknown"`. Those outcomes are plausible and safe. This review has not shown that the baseline accepts a false equality on such an input. The finding is that its promised implication

$$\text{ExactAlgebraicQ}[e] = \text{True} \implies e \in \overline{\mathbb{Q}} \quad (8)$$

is not established by the implementation.

4.3 A conservative repair

The revision accepts indexed univariate roots only when the defining function produces a nonconstant polynomial with rational coefficients and the index is an integer in the degree range. The supported optional root-isolation flag is also checked. An `AlgebraicNumber` must have an admitted generator and a Gaussian-rational coefficient list. Plus, Times, rational Power, and Gaussian rationals retain their previous recursive treatment.

This is deliberately a *sufficient grammar*. It rejects some genuinely algebraic objects, including more general triangular representations and roots with algebraic coefficients that have not been reduced to a rational-polynomial form. Such inputs may first be normalized with a bounded `RootReduce` and then rechecked. The revision does not claim to decide algebraicity of arbitrary Wolfram expressions. Root-defining functions are ordinary Wolfram code, so this validator is not a sandbox for hostile evaluation or side effects.

4.4 Make all three equality statuses honest

The baseline evaluates a difference numerically at 40 digits and, when its reported accuracy and magnitude exceed thresholds, returns `"Different"`. This branch never accepts a candidate, which is an important safety distinction. But it is not the same as an exact proof of inequality, and its existence conflicts with the public exact-decision wording.[1], lines 186–212

The proposed revision keeps the option for compatibility but changes its role. Numerical evidence may increment a diagnostic statistic; it neither returns `"Different"` nor prevents exact

certification. Exact zero from `RootReduce` proves equality. A rational or Gaussian-rational nonzero normal form proves inequality. A validated polynomial root with nonzero constant coefficient cannot be zero. Remaining cases go through the exact algebraic method; interruption or uncertainty produces **"Unknown"**. [10, 9]

The record for an accepted candidate now identifies which exact equality route succeeded. This is useful provenance, but it is still not an independent proof object: both discovery and certification depend on the same kernel.

Proposition 4.1 (Conditional soundness of the acceptance gate). *Assume that domain admission implies a finite exact algebraic value and that all equality-oracle answers **"Equal"** are correct. If a gate returns the incumbent unchanged or replaces it only by a cheaper radical expression certified equal to the original target, then every sequence of successful stage transitions preserves equality to that target and never increases cost.*

Proof. The initial incumbent equals the target. An unchanged transition preserves both properties. A replacing transition establishes equality to the original target and strict cost decrease by its explicit tests. Induction proves the claim. The hypotheses explain why domain validation and exact-oracle semantics belong to the trusted core, whereas discovery formulas need not be trusted. $\square$

5 Search-state defects and concrete witnesses

5.1 Later no-op stages can erase earlier success

In `powerProposals`, the assignments

```
best = negativeRadicandCandidate[target, rho, p, q];
...
best = kummerCandidates[target, rho, p, q];
```

do not pass `best`. Both callees initialize or fall back to `target`. A later no-op is thus interpreted as “replace the incumbent by the original input,” rather than “retain the incumbent.”[1], lines 435–464,598–620

An important qualification is that the specialized fast kernels usually return depth-one candidates, after which `powerProposals` exits early. It would therefore overstate the problem to claim that every successful quadratic simplification is subsequently lost. The exposed case is a *partial* improvement, notably one found while separating a negative radicand.

Consider the positive numbers

$$t = 11 - 2\sqrt{29}, \quad w = \sqrt{55 - 10\sqrt{29}}, \quad (9)$$

$$\rho = 16 - 2\sqrt{29} + 2w, \quad B = \sqrt{5} + \sqrt{t}. \quad (10)$$

Since $11 > 2\sqrt{29}$, $t > 0$, and $w = \sqrt{5t}$. Hence

$$B^2 = 5 + t + 2\sqrt{5t} = \rho, \quad B > 0. \quad (11)$$

This is the partial-denesting example discussed in the square-root literature.[6] The negative-radical target is

$$E = \sqrt{-\rho} = iB. \quad (12)$$

Its original presentation has depth three, while iB has depth two. If recursive processing of the positive modulus discovers B , the negative stage can accept iB . A subsequent Kummer stage which returns its original target discards that certified improvement.

There is also a field-theoretic reason that a linear-factor search need not rediscover the improvement. Put $F = \mathbb{Q}(\sqrt{29})$ and $K = F(w)$. This is a real field, so $iB \notin K$. Moreover $B = \sqrt{5}(1 + w/5)$, and $\sqrt{5} \notin K$: 5 is not a square in F , and $5/(5t) = 1/t$ is not a square in F , since $N_{F/\mathbb{Q}}(t) = 5$ is not a rational square. Thus even the positive square root is outside the displayed radicand field. This does not prove how every kernel factorization is represented; it explains why retaining independent discoveries is mathematically necessary.

The supplied white-box regression injects the partial modulus result and an empty linear-factor result. It tests the stage interface directly without pretending that default CAS discovery was observed. The repair is simply to pass an incumbent through both functions and to return it on every no-op path.

5.2 Useful index reduction does not require recursive progress

The baseline takes a root γ of $x^k - \rho$, writes $q = kr$, and recursively processes $\gamma^{1/r}$. It offers the result only if recursion changed the expression and reduced radical depth.[1], lines 443–452

Both requirements are too strong. Set

$$\rho = 3 + 2\sqrt{2} = (1 + \sqrt{2})^2, \quad E = \rho^{1/4} = \sqrt{1 + \sqrt{2}}. \quad (13)$$

The reduced expression already has a smaller presentation even though both forms have radical depth two and two nonintegral-power nodes. The coefficient and multiplication in $2\sqrt{2}$ disappear. With recursion disabled, the old condition necessarily rejects this opportunity.

The example fits the Kummer search exactly. For $k = 2$, the radical field $\mathbb{Q}(\sqrt{2})$ contains $\gamma = 1 + \sqrt{2}$. It does not contain the positive square root of γ : the conjugate $1 - \sqrt{2}$ is negative, while the conjugate of a square in this real quadratic field must be nonnegative. Thus asking for a linear factor of $x^4 - \rho$ first does not make the $k = 2$ step redundant.

The revised stage first offers the unmodified reduced presentation, with the necessary root-of-unity phases and the original numerator exponent. It then optionally recurses and offers those results too. The global acceptance gate, not a separate depth-only predicate, decides whether each proposal is useful. This also admits equal-depth improvements in node count or coefficient size.

5.3 A failed bounded search is not a reusable negative fact

The baseline memo key is the target expression alone. On completion it stores **best**, including the case **best** === **target**; a later visit returns that entry immediately. Recursive calls, however, use fewer remaining recursion levels and divide the trial allowance by four. Local operation failures can also occur without exhausting the session deadline.[1], lines 430–433,628–639

Suppose a target is first visited near the recursion limit. It may return unchanged because the remaining search is weak, and that unchanged result is cached. A later shallower visit has more opportunities, but the memo lookup prevents them from being tried. The cache has confused “this search found nothing” with “nothing useful remains to be found.”

One repair is a resource-aware search-state key. Another is to store only certified successes and use them as incumbents rather than terminal answers. The revision chooses the latter. It also maintains a path-local in-progress set, so removing negative entries does not invite immediate recursive cycles. A revisit on the current path returns the best already known representation; a later visit off that path is allowed to search again.

This policy can perform repeated unsuccessful work. That is a conscious tradeoff for simpler correctness of search-state reuse. A more advanced implementation could store an incumbent plus a resumable frontier and a clearly specified coverage descriptor, but an undifferentiated memoized expression is insufficient.

6 Resource contracts, option dispatch, and metric details

6.1 An overall timeout does not enforce every local timeout

The header promises a bounded region for every expensive kernel operation. The source nevertheless performs `FactorInteger` directly while building multisurd bases and discriminant-prime lists. It also expands an ansatz square and normalizes some extracted linear roots outside `bounded`.^[1], lines 357, 364,425–426,564

These calls are usually inside the *outer* session constraint. Therefore it would be incorrect to say that the code has no time limit at all. The defect is that `"OperationTime"` does not govern the operations in question. A full factorization of a large discriminant can consume most of the session, even though only eight resulting primes will be retained. Taking a prefix *after* complete factorization does not bound factorization effort.

The revision wraps these explicit calls and treats failure as absence of that proposal source. It also bounds the combined shape-specific proposal call and rejects oversized candidates before polishing. These are practical safeguards, not a proof that expression evaluation has constant overhead or that every internal arithmetic step is separately interruptible.

6.2 The discriminant batch bound is misplaced

In the old prime-power loop, the batch counter is checked before entering each prime, but not before each exponent:

```
Do[If[batch >= 24, Break[]];
  Do[batch++; admit[m prime^e], {e, 1, q - 1}],
  {prime, primes}]
```

For $q = 24$, one prime contributes 23 proposals. Because $23 < 24$, a second prime is entered and contributes another 23. The batch reaches 46. For $q = 32$, the first prime alone reaches 31. Among $2 \leq q \leq 32$ and up to eight primes, the largest such overshoot is 46. The independent loop-model checks exhaust this finite range.^[1], lines 565–572

The queue's separate admission guard still prevents more than `"MultiplierCap"` entries from being admitted. It is the local generation budget, not the global queue bound, that fails. The revision checks all relevant bounds in the exponent loop and exposes the old constant as `"DiscriminantBatchCap"`, defaulting to 24. A value of zero disables that expansion source without disabling seed multipliers.

6.3 Budgeted work should include preflight and report metrics

The original outer region contains the pass loop, but input scans before it and `RadicalCost` evaluations used to assemble the final report lie outside it. A zero time budget therefore still permits full cost traversal of the input. For large trees, this contradicts a literal whole-call budget.^[1], lines 692–728

In the revision, input scans and initial/candidate costs are performed inside the outer region. The committed snapshot carries its already-computed cost. When the initial cost was never computed, the report uses `Missing["NotComputed"]` instead of performing expensive post-timeout work. The zero-budget path does not scan the expression merely to populate cosmetics.

Ordinary argument evaluation, option resolution, final association construction, and front-end serialization are not claimed to be covered. A real hard boundary requires a parent process supervising a worker kernel; the parent can enforce wall-clock and operating-system memory

limits and recover a previously serialized checkpoint. That is an architectural extension, not something accomplished by adding another `TimeConstrained` around the same kernel expression. Wolfram documents interrupt granularity and abort protection for time constraints, and states that their accounting concerns main-kernel CPU time rather than all threads/processes. Memory constraints are likewise not a process-RSS sandbox. The revision's report therefore describes its limits as cooperative kernel limits.[11, 12]

6.4 Degree limits are output filters, not complexity certificates

The package computes a minimal polynomial and then checks its degree against `"MaxDegree"`. That is a sensible acceptance filter, but a polynomial of unacceptable degree may already have been expensive to compute. The same observation applies to extension construction and algebraic normalization. The revision retains this behavior rather than falsely promising a cheap pre-computation degree bound.

A future preflight may multiply upper bounds for independent radical degrees, reduce known square-class relations, and stop when the bound becomes too large. Such a bound must be conservative: naive multiplication often dramatically overestimates the true field degree. A pessimistic bound is useful for resource policy, not a reason to claim that a denesting is mathematically unavailable.

6.5 Malformed options should reach the validator

The validator checks that option items are rules and returns a structured `Failure`. But the baseline public definitions use `OptionsPattern[]` for the tail. A non-rule argument such as 123 need not match those definitions, so it may remain an unevaluated public call rather than reaching the validator. This is a dispatch gap between the documented API and its error path. [1], lines 120–138,736–742

The proposed public definitions accept an arbitrary option tail and pass it to the same validator. The more specific legacy Boolean overload remains. Unknown rule names, invalid values, and malformed non-rule tails consequently share a consistent error mechanism. Missing required positional arguments remain an ordinary arity issue; this change does not purport to define every malformed call imaginable.

6.6 Cost accounting should be direct

The original radical-node counter counts all powers and then subtracts powers inside opaque algebraic objects. A pruned recursive traversal is simpler and cannot double-subtract a radical if an opaque representation contains another opaque object. This review treats the latter as a representation-dependent robustness issue, not a demonstrated negative count for a particular native canonical input.[1], lines 173–181

There is a more direct coefficient issue: exact complex numbers are atomic to ordinary expression traversal. Searching for integer/rational atoms therefore does not reliably price their real and imaginary components. The replacement `bitSize` handles `Complex` explicitly, then recurses through other non-atomic payloads. Opaque counts and radical-node counts prune opaque bodies; bit size still prices their stored data. The revised cost documentation states this distinction rather than leaving it implicit.

Even a perfectly implemented lexicographic metric is noncompositional under all Wolfram evaluations. A local simplification can change sharing, cancellation, or a parent's final representation. The package still checks whole-pass cost before commitment. A future DAG-aware metric or a

small Pareto frontier could improve coverage, but neither is implemented here.

7 Verification of the retained algebraic kernels

This section records the mathematical contracts that the implementation should preserve. It also separates a formula being a valid *proposal* from it already being the correct principal value. The exact gate remains the final authority for the latter.

7.1 Direct square-root denesting

Let $a, b, c \in \mathbb{Q}$, $c > 0$, $\sqrt{c} \notin \mathbb{Q}$, and suppose $\rho = a + b\sqrt{c} > 0$. Put $D = a^2 - b^2c$. When $D \geq 0$ and $s = \sqrt{D} \in \mathbb{Q}$, set

$$u = \frac{a + s}{2}, \quad v = \frac{a - s}{2}. \quad (14)$$

Under $a > 0$, both u and v are nonnegative, because $s \leq a$. Moreover $uv = b^2c/4$, so

$$(\sqrt{u} + \operatorname{sgn}(b)\sqrt{v})^2 = a + b\sqrt{c}. \quad (15)$$

Since $u \geq v$, the displayed candidate is nonnegative even when $b < 0$. It is therefore the principal square root. The source absorbs $|b|$ into the radicand before applying the recipe, which changes its internal representation but not this argument. The norm/discriminant mechanism is the familiar quadratic denesting criterion discussed in the literature.[6, 5]

7.2 Indirect denesting with a fourth root

Suppose $D < 0$ and $b > 0$. Define

$$h = \sqrt{b^2 - a^2/c}. \quad (16)$$

When $h \in \mathbb{Q}$, let $u = (b + h)/2$ and $v = (b - h)/2$. Here $0 \leq h \leq b$, so $u, v \geq 0$, and $uv = a^2/(4c)$. Consequently,

$$\left[c^{1/4} (\sqrt{u} + \operatorname{sgn}(a)\sqrt{v}) \right]^2 = \sqrt{c} \left(b + \frac{a}{\sqrt{c}} \right) \quad (17)$$

$$= a + b\sqrt{c}. \quad (18)$$

The candidate is nonnegative because $u \geq v$. This proves the source's recipe. The rational-square condition is

$$h^2 = -D/c, \quad h \in \mathbb{Q} \iff -cD \text{ is a rational square.} \quad (19)$$

It is not the condition that $-D$ alone be a rational square. For instance,

$$\sqrt{4 + 3\sqrt{2}} = 2^{1/4}(1 + \sqrt{2}), \quad (20)$$

where $D = -2$ and $-cD = 4$. This distinction is already emphasized in the survey and preceding correction and is retained here.[3, 2, 6]

7.3 Principal roots on the negative real axis

Wolfram rational powers use a principal complex value. For $r > 0$ and rational p/q in lowest terms,

$$(-r)^{p/q} = e^{i\pi p/q} r^{p/q} = (-1)^{p/q} r^{p/q}. \quad (21)$$

For odd q , this generally differs from the signed real q th root convention. In particular,

$$(-7 - 5\sqrt{2})^{1/3} = (-1)^{1/3}(1 + \sqrt{2}), \quad (22)$$

not $-1 - \sqrt{2}$. A power identity alone does not choose the correct root: all q root-of-unity multiples have the same q th power. The implementation's phase enumeration and final exact comparison against the original principal expression are therefore essential.

Similarly, unrestricted transformations such as $(uv)^{1/q} = u^{1/q}v^{1/q}$ or $(u^k)^{1/k} = u$ must not be made into trusted rewrite rules. They can be used to organize a finite candidate orbit whose members are checked individually.

7.4 Odd roots in a quadratic field

The cubic kernel admits a clean generalization. Suppose

$$\beta = A + B\sqrt{c}, \quad A, B, c \in \mathbb{Q}, \quad (23)$$

with conjugate $\bar{\beta} = A - B\sqrt{c}$. Define the trace and norm

$$T = \beta + \bar{\beta} = 2A, \quad n = \beta\bar{\beta} = A^2 - cB^2. \quad (24)$$

Both roots satisfy $x^2 - Tx + n = 0$. Thus the polynomial sequences

$$D_0 = 2, \quad D_1 = T, \quad D_j = TD_{j-1} - nD_{j-2}, \quad (25)$$

$$S_0 = 0, \quad S_1 = 1, \quad S_j = TS_{j-1} - nS_{j-2} \quad (26)$$

satisfy

$$D_j = \beta^j + \bar{\beta}^j, \quad (\beta - \bar{\beta})S_j = \beta^j - \bar{\beta}^j. \quad (27)$$

These identities follow by the same second-order recurrence and the initial values. They also give the polynomial identity

$$D_j(T, n)^2 - (T^2 - 4n)S_j(T, n)^2 = 4n^j. \quad (28)$$

It is a polynomial identity even when the two quadratic roots coincide, since it holds for indeterminate distinct roots and hence identically.

Now let $q \geq 3$ be odd and suppose we seek a root inside $\mathbb{Q}(\sqrt{c})$ of

$$\beta^q = a + b\sqrt{c}, \quad b \neq 0. \quad (29)$$

Taking norms gives the necessary condition

$$n^q = a^2 - b^2c. \quad (30)$$

The rational candidate for n is the signed real q th root of the right-hand side. For every rational root T of

$$D_q(T, n) - 2a = 0, \quad (31)$$

form

$$\beta = \frac{T}{2} + \frac{b}{S_q(T, n)}\sqrt{c}. \quad (32)$$

Proposition 7.1 (Trace–norm reconstruction). *Under the assumptions above, if $n, T \in \mathbb{Q}$ satisfy the norm equation and (31), then $S_q(T, n) \neq 0$ and the value in (32) satisfies $\beta^q = a + b\sqrt{c}$. Conversely, every such root in $\mathbb{Q}(\sqrt{c})$ supplies such a pair (T, n) .*

Proof. Substituting the equations for D_q and n^q into (28) yields

$$(T^2 - 4n)S_q(T, n)^2 = 4b^2c \neq 0. \quad (33)$$

In particular $S_q \neq 0$. Writing $A = T/2$ and $B = b/S_q$ gives $A^2 - cB^2 = n$. Therefore the reconstructed quadratic pair has trace T and norm n , and (27) shows $\beta^q = D_q/2 + B\sqrt{c}S_q = a + b\sqrt{c}$. The converse follows by taking the trace and norm of the assumed root. $\square$

For $q = 3$,

$$D_3 = T^3 - 3nT, \quad S_3 = T^2 - n, \quad (34)$$

which recovers the existing cubic implementation. For $q = 5$,

$$D_5 = T^5 - 5nT^3 + 5n^2T, \quad S_5 = T^4 - 3nT^2 + n^2. \quad (35)$$

The revision uses rational linear factors of the trace polynomial rather than trying to solve an arbitrary quintic or septic by radicals. By default the extension is enabled through index seven. This is a bounded additional proposal kernel, not a completeness statement for all odd radicals: roots requiring a larger field are outside its scope. The original cubic kernel is retained.

7.5 The Honsbeek quartic proposal is algebraically sound

Let $A^3 = a$, $B^3 = b$, and set

$$N = B^2 + sAB - \frac{s^2A^2}{2}, \quad D = b - s^3a. \quad (36)$$

Expanding and reducing $A^3 = a$, $B^3 = b$ gives

$$N^2 = (b - s^3a)B + A \left(2sb + \frac{s^4a}{4} \right), \quad (37)$$

$$N^2 - (A + B)D = \frac{A}{4} [a(s^4 + 4s^3) + b(8s - 4)]. \quad (38)$$

Thus every rational solution of

$$s^4 + 4s^3 + 8(b/a)s - 4b/a = 0 \quad (39)$$

with $a \neq 0$ and $D \neq 0$ gives candidates $\pm N/\sqrt{D}$ whose squares equal $A + B$. The source restricts to $D > 0$ and offers both signs. The exact gate selects the original principal square root.[1], lines 325–343

For $A = \sqrt[3]{5}$, $B = -\sqrt[3]{4}$, one may take $s = -2$, giving $D = 36$ and

$$\sqrt{\sqrt[3]{5} - \sqrt[3]{4}} = \frac{\sqrt[3]{2} + \sqrt[3]{20} - \sqrt[3]{25}}{3}. \quad (40)$$

The positive sign is immediate because $\sqrt[3]{2} + \sqrt[3]{20} > 3$ while $\sqrt[3]{25} < 3$. This familiar example is also present in the repository literature.[5, 7] The independent test suite verifies (38) symbolically in a polynomial quotient ring, rather than merely checking decimal approximations.

7.6 Polynomial inverses for rationalization

If a nonzero algebraic denominator d satisfies

$$a_0 + a_1d + \cdots + a_nd^n = 0, \quad a_0 \neq 0, \quad (41)$$

then

$$d^{-1} = -\frac{a_1 + a_2d + \cdots + a_nd^{n-1}}{a_0}. \quad (42)$$

This is exactly the Horner expression used in `rationalizeRaw`. The source checks that coefficients are rational, excludes zero constant coefficient, and verifies $d \cdot d^{-1} = 1$ before using the inverse. There is no missing sign or off-by-one error in the formula.[1], lines 261–279

The result need not be cheaper, which is why the standalone rationalization API has a different cost contract from a denesting proposal. When a symbolic numerator is present, multiplying by a certified inverse of a fixed nonzero numeric denominator is a congruence step; it is not a license to cancel arbitrary symbolic factors whose zero sets were not tracked.

8 The sparse multisurd ansatz: a proved limitation

8.1 A counterexample inside a small field

The baseline multisurd kernel forms two trial sets: 1 together with visible prime factors, and then those values together with the visible radicands. It solves for a rational linear combination of the corresponding square roots. These are useful small ansatzes, but they do not constitute a basis for all possible denestings.[1], lines 345–375

Take

$$B = \sqrt{6} + \sqrt{10} + \sqrt{21}. \quad (43)$$

Direct expansion gives

$$B^2 = 37 + 4\sqrt{15} + 6\sqrt{14} + 2\sqrt{210}. \quad (44)$$

The old largest ansatz uses

$$U = \{1, 2, 3, 5, 7, 14, 15, 210\}. \quad (45)$$

It contains none of the square classes 6, 10, or 21. Distinct square classes have linearly independent square roots over $\mathbb{Q}$ inside a multiquadratic field. Consequently neither B nor $-B$ lies in the rational span of $\{\sqrt{u} : u \in U\}$. Since the only roots of $x^2 - B^2$ are $\pm B$, the ansatz has no solution.

For completeness, the independence statement follows from character orthogonality. Choose independent square classes generating a field containing all the roots. Each root belongs to a distinct one-dimensional eigenspace for the sign-changing automorphism group. Applying the projector for one character to a rational linear relation isolates its coefficient, which must therefore vanish. This argument also motivates the replacement kernel below.

The conclusion is about the *multisurd kernel*. Kummer, generic CAS, or multiplier proposals might still solve the same whole input. This review does not turn a demonstrated local search-space omission into an unmeasured end-to-end failure claim.

8.2 Closing the visible prime basis is still not enough

A tempting repair is to include every product of the visible radicand primes. That helps, but it does not solve the general issue. For example,

$$\left(\sqrt{10} + \sqrt{15} + \sqrt{35}\right)^2 = 60 + 10\sqrt{6} + 10\sqrt{14} + 10\sqrt{21}. \quad (46)$$

The prime 5 is absent from the radicands 6, 14, and 21. It occurs in coefficients, and the square root lies in a square-class coset not contained in the field obtained from those radicands alone.

In (44), the radicand field is $K = \mathbb{Q}(\sqrt{14}, \sqrt{15})$. The classes supporting B lie in

$$6\langle 14, 15 \rangle \subset \mathbb{Q}^\times / \mathbb{Q}^{\times 2}, \quad (47)$$

not necessarily in the subgroup itself. Multipliers can sometimes discover this missing coset; a direct reconstruction method can find it without first guessing its representative.

9 A bounded character-reconstruction kernel

9.1 Square-class basis without an explicit prime-factor basis

Suppose

$$\rho = a_0 + \sum_j a_j \sqrt{d_j}, \quad a_j \in \mathbb{Q}, \quad d_j \in \mathbb{Z}_{>0}. \quad (48)$$

Let H be the subgroup of $\mathbb{Q}^\times / \mathbb{Q}^{\times 2}$ generated by the d_j , and let r be its rank. Then $K = \mathbb{Q}(\sqrt{h} : h \in H)$ has degree $m = 2^r$.

The new kernel constructs representatives of H incrementally. Starting with $\{1\}$, a new radicand d is already represented precisely when d/h is a rational square for some existing representative h . Otherwise d becomes a new generator and the representative list is doubled by multiplication by d . The rank cap is checked before that doubling.

There is no explicit `FactorInteger` call in this basis construction. The Wolfram implementation uses exact rational-square tests involving square roots; it does not promise that the kernel never factors internally while simplifying them. An integer-square-root test on the numerator and denominator would be a possible further low-level optimization.

9.2 Conjugates and the finite transform

Index H and the automorphisms of K by $\mathbb{F}_2^r$. Let

$$\chi_g(\sigma) = (-1)^{g \cdot \sigma}, \quad \lambda_\sigma = \sqrt{\sigma(\rho)} \quad (49)$$

with the principal square root at each embedding. Enumerate signs $\epsilon_\sigma \in \{\pm 1\}$, fixing $\epsilon_0 = 1$. For each choice compute

$$F_g = \frac{1}{m} \sum_\sigma \chi_g(\sigma) \epsilon_\sigma \lambda_\sigma. \quad (50)$$

If every F_g^2 is rational, each F_g can be represented by one signed square root of a rational number, with the sign determined by exact comparison. The candidate is then

$$\sum_g F_g. \quad (51)$$

Orthogonality of the characters shows that this sum is exactly λ_0 : all columns except the identity column cancel. Nevertheless, the implementation also runs the final exact equality gate. A proof of the algebraic construction should not become a reason to bypass a robust implementation boundary.

9.3 Why this captures the missing coset

Lemma 9.1 (Single-coset support). *Let $K/\mathbb{Q}$ be multiquadratic. Suppose $\beta \neq 0$, $\beta^2 \in K$, and β lies in some finite multiquadratic extension of $\mathbb{Q}$. Then $\beta \in \sqrt{d}K$ for some $d \in \mathbb{Q}^\times$.*

Proof. Choose a finite multiquadratic field M containing both K and β . For $\tau \in \text{Gal}(M/K)$, the equality $(\tau\beta)^2 = \beta^2$ gives $\tau\beta = \pm\beta$. These signs form a character of $\text{Gal}(M/K)$, because $\beta \neq 0$. The Galois group of $M/\mathbb{Q}$ is a finite vector space over $\mathbb{F}_2$, so that character extends to the whole group. Its one-dimensional eigenspace is generated by $\sqrt{d}$ for a rational square class d . The quotient $\beta/\sqrt{d}$ is fixed by $\text{Gal}(M/K)$ and hence lies in K . $\square$

Theorem 9.2 (Restricted completeness of exhaustive reconstruction). *Ignore computational limits and suppose exact algebraic arithmetic and equality are available. For $\rho \in K$, the sign enumeration in (50) finds a representation of the principal $\sqrt{\rho}$ as a sum of square roots of rational numbers whenever such a representation exists. Signed rational radicands, and thus complex square roots, may be allowed. This statement does not include arbitrary fourth roots or higher-index radicals.*

Proof. The zero case is trivial. A finite sum of rational square roots lies in a finite multiquadratic extension. By the lemma, $\beta = \sqrt{\rho} = \sqrt{d}u$ with $u \in K$. If $\sqrt{d} \in K$, absorb it into u and use $d = 1$. Otherwise each automorphism of K extends to $K(\sqrt{d})$ while fixing $\sqrt{d}$.

Expand $u = \sum_g c_g \sqrt{h_g}$ in the square-class basis of K , where $c_g \in \mathbb{Q}$. At each embedding, the image of β is one of the two square roots of $\sigma(\rho)$, so it equals $\epsilon_\sigma \lambda_\sigma$ for some enumerated sign choice. At the identity embedding the sign is $+1$. Character projection then gives $F_g = c_g \sqrt{d h_g}$, and $F_g^2 = c_g^2 d h_g \in \mathbb{Q}$. Thus that sign choice passes the rational-square tests and reconstructs β . Conversely, whenever the tests pass, selecting the exact signs of the square roots gives the F_g themselves; orthogonality proves (51) equals the principal target. $\square$

The theorem is a mathematical guarantee for an exhaustive restricted procedure, not a guarantee for a timed execution. It is also not a new general denesting theorem: it is an explicit character-based formulation suitable for this implementation. The preceding review already pointed toward field bases and character projections; this revision turns a small-rank form into executable proposal code.[2]

9.4 An explicit reconstruction table

For (43), write $a = \sqrt{6}$, $b = \sqrt{10}$, $c = \sqrt{21}$. Choose generators 14 and 15 and extend their sign changes while fixing $\sqrt{6}$. The principal square roots of the four conjugates are

$$(\lambda_0, \lambda_1, \lambda_2, \lambda_3) = (a + b + c, a + b - c, a - b + c, -a + b + c). \quad (52)$$

All four displayed values are positive. The only nontrivial triangle inequality is $a + b > c$, which follows from $(a + b)^2 = 16 + 4\sqrt{15} > 21$; the other two follow already from $a^2 + c^2 > b^2$ and $b^2 + c^2 > a^2$.

Use the sign vector $(1, 1, 1, -1)$. The Hadamard transform gives

$$(F_0, F_1, F_2, F_3) = (\sqrt{6}, \sqrt{21}, \sqrt{10}, 0), \quad (F_0^2, F_1^2, F_2^2, F_3^2) = (6, 21, 10, 0). \quad (53)$$

The missing square classes are recovered directly. With this generator order and the implementation's binary sign enumeration, this is the second sign pattern, not an expensive search through all possible rational multipliers. The independent checks verify this example and 48 additional exact rank-two coset examples using separate multiquadratic arithmetic and certified rational intervals for signs.

9.5 The cost is exponential in field degree

There are $2^{m-1} = 2^{2^r-1}$ sign patterns. The distinction between rank and field degree is operationally important:

Square-class rank r	Field degree m	Sign patterns
1	2	2
2	4	8
3	8	128
4	16	32,768

The revision defaults to rank two and eight patterns. Rank three with 128 patterns is an explicit opt-in; rank four is accepted only with a deliberately chosen pattern budget and remains potentially expensive. Each algebraic reduction and sign certificate is also subject to the shared limits.

The older sparse ansatz remains as a fallback. Preserving it avoids throwing away a useful fast heuristic for inputs outside the new rank cap. The new kernel supplements rather than replaces the larger Kummer and multiplier machinery.

10 The proposed source revision

10.1 A standalone file, not an undocumented patch layer

`StradFixed3.wl` is a complete loadable package in `RadicalDenest3`. It does not require loading version two first and does not modify the user's repository. The archive also includes `StradFixed2.original.wl`, the exact reviewed baseline, `revision_definitions.wl`, and `build_revision.py`.

The builder verifies the baseline's Git-blob identity, applies anchored changes, removes replaced function definitions, appends the new private definitions before closing the package, and writes the standalone file. Every replacement anchor has an expected occurrence count, so the builder fails rather than silently applying a review to a different source version. A unified diff is included for human inspection.

The original low-degree and general-search algorithms remain present. Additional changes include fresh multisurd coefficient symbols, early size checking before polishing, exact equality-method labels in accepted-proposal records, and explicit report wording about cooperative resource limits and certificate scope. These are small but useful measures against accidental coupling and misleading diagnostics.

10.2 Behavior intentionally changed

The changes are not perfectly backward compatible. A valid algebraic value in an unsupported `Root` representation can now be rejected by the conservative grammar. Numeric prefiltering defaults to false, and setting it true no longer produces an unproved inequality status. Gaussian bit accounting and opaque pruning can change cost tie-breaks. A zero-budget report can contain missing cost fields. Malformed option tails now produce structured failures instead of remaining unevaluated.

These changes are documented because they are contract corrections, not incidental implementation details. Existing callers that compare exact output syntax should instead check equality and the desired presentation property, unless a specific normal form is part of their own API.

10.3 New options

Option	Default	Meaning
"MaxCharacterRank"	2	Maximum independent square-class rank for the new reconstruction kernel. Zero disables it; values above four are rejected.
"MaxCharacterPatterns"	8	Maximum sign patterns attempted. Zero disables character reconstruction. A truncated enumeration is not a negative mathematical answer.
"MaxOddIndex"	7	Largest odd index admitted by the additional quadratic-field trace-norm kernel. The existing cubic path is retained.
"DiscriminantBatchCap"	24	Maximum prime-power/mixed-product proposal attempts generated by one discriminant expansion. The global queue and trial limits remain separate.

All existing options remain. "MaxTrials" continues to be per island, not a global counter for the entire call; recursive invocations continue to use smaller trial allowances. The overall session deadline and memory constraint remain shared. Those distinctions are intentional and should not be reported as regressions.

10.4 Usage

```
Get["StradFixed3.wl"];

RadicalDenest3'DenestCore[
  Sqrt[37 + 4 Sqrt[15] + 6 Sqrt[14] + 2 Sqrt[210]],
  "MaxTrials" -> 0
]

RadicalDenest3'DenestReport[
  (41 + 29 Sqrt[2])^(1/5),
  "Trace" -> True,
  "MaxTrials" -> 0
]

RadicalDenest3'DenestRadicals[
  expression,
  "AllLevels" -> True,
  "MaxCharacterRank" -> 3,
  "MaxCharacterPatterns" -> 128
]
```

The first call targets $\sqrt{6} + \sqrt{10} + \sqrt{21}$ and the second targets $1 + \sqrt{2}$. These are mathematical expected results and native regression fixtures; they are not a transcript of a kernel session performed for this review. Fully qualified names are useful when loading versions two and three side by side.

11 Validation, reproducibility, and remaining qualification

11.1 What actually ran

The independent suite completed 396 checks with zero failures under Python 3.13.5 and SymPy 1.14.0. The JSON output records every check. Its coverage includes:

- the baseline Git-object hash; symbolic Honsbeek, trace-norm, and polynomial inverse identities; exact odd-root reconstruction instances;
- exact square-class arithmetic, the sparse-ansatz witness, Hadamard orthogonality, and exhaustive rank-two reconstruction witnesses;
- the finite discriminant-batch model, separate global-admission caps, partial-denesting/index-reduction identities, and lexical/source checks.

Polynomial identities use exact algebra. Sign isolation for the multiquadratic examples uses rational outward bounds derived from integer square roots on a 10^{-60} grid. No floating-point agreement is used as a proof of equality. The word “passed” applies to these particular checks only.

The source checks verify balanced delimiters after removing strings and nested comments and test for the intended critical source patterns. They cannot detect all Wolfram semantic errors. In particular, they do not establish option precedence, message behavior, native factorization choices, interrupt behavior, or actual end-to-end runtime.

11.2 The native regression suite

The 40 supplied Wolfram tests cover conservative root admission, exact equality, wrong branches, Gaussian coefficient cost, malformed options, disabled budgets, quadratic/cubic/fifth-root examples, the missing-coset example, rationalization, restricted host traversal, callback rejection, index reduction without recursive progress, incumbent preservation, negative-cache removal, cycle cutting, and certificate replay. Some are black-box tests; others deliberately stub discovery functions so that a control-flow contract can be tested independently of CAS heuristics.

Run them in a native kernel with

```
wolframscript -file run_tests.wls
```

The runner writes `native_results.json` containing the kernel version and success/failure counts, and exits with a nonzero status on failed tests. That file is deliberately absent from this delivery: manufacturing it would imply an execution that did not occur.

The tests are a starting qualification suite, not a claim that all behavior of the preceding implementation has been covered. The repository’s original `StradFixed2Tests.wlt` should also be run against the revision with public contexts adapted, treating intentional API changes separately. Prior reported test success belongs to the prior implementation and environment; it cannot be inherited by a source revision.^[2]

11.3 What should be measured next in a native environment

The first native run should inspect all messages, not merely count equality checks. A candidate that simplifies correctly only after emitting an unexpected message may be turned into `$Failed` by a surrounding `Check`. Deterministic tests should then be followed by generated exact families, varying signs, numerator exponents, scalar factors, and repeated occurrences at different recursion depths.

For timing, record wall time and the package’s counters separately. Compare cold and warm kernels because algebraic and factorization caches affect search cost. Measure default rank-two character reconstruction before raising the rank cap. Keep equality-preservation success distinct from denesting coverage: a result can be correct yet unchanged, or correctly partially denested without reaching the best known presentation.

The honest release status is therefore *proposed research revision with independently checked mathematics and supplied native tests*. It is not yet a production-qualified replacement, and it is not a proof that the new defaults outperform the old defaults on every input family.

12 Further improvements, in priority order

12.1 Make the trusted core smaller and more explicit

The domain recognizer, equality oracle, and commit mechanism should be separable from discovery. A future certificate structure could contain the exact input and output values, an annihilating polynomial, isolating data for the selected embedding, and an independently replayable equality witness. A record saying which kernel function returned true is useful audit metadata, but it is not such a certificate.

A structural proof graph would distinguish accepted proposals, local replacements, congruence rebuilding, and committed passes. The current bounded certificate sample can contain proposals from an uncommitted pass. Its revised description says so explicitly. It should not be interpreted as the exact derivation of the published final result.

12.2 Cache mathematical state separately from search coverage

Positive-only caching avoids the immediate defect but leaves performance on the table. A more capable cache entry would store a certified incumbent plus a frontier of untried multipliers, divisors, and sign patterns. It could also record which field presentation was used, which algorithms completed, and what resources were available. An interrupted factorization is not a completed negative field-membership test, and an exhausted prefix of sign patterns is not an exhausted search space.

Canonical algebraic keys could improve deduplication, but normalizing every cheap multiplier with `RootReduce` may dominate the search. One reasonable compromise is cheap syntactic deduplication first and bounded algebraic canonicalization only for promising expensive trials. No such optimization should be justified by calling an expanded-expression string a canonical algebraic value.

12.3 Separate field discovery from radical presentation

The current radical-extension list can contain redundant generators. A reduced primitive-element representation, together with maps back to display radicals, would make repeated field operations less wasteful. Square-class rank reduction is the simplest instance; higher-index Kummer classes require a more general representation.

A low minimal-polynomial degree is useful evidence that a presentation might be obtainable, but it is not itself a denesting formula or a completeness certificate. Similarly, `ToRadicals` is a proposal mechanism with documented scope, not an unrestricted denesting oracle.^[13] The repository’s survey and Landau discussion emphasize why roots of unity and the chosen base field cannot be treated as free without changing the optimization problem. [3, 7]

12.4 Preserve useful partial work without weakening publication

The current complete-pass checkpoint is safe but coarse. A timeout late in a large symbolic-host traversal can discard many independently certified local improvements from that pass. Publishing a partially rebuilt tree safely requires tracking the original context and already certified replacements, not simply returning whatever local variable happened to be last assigned.

A persistent expression tree or a zipper with a replacement log would permit finer checkpoints. A supervising process could store those checkpoints durably. The revision retains complete-pass publication because it is easier to audit; it does not disguise that decision as optimal recovery behavior.

12.5 Optimize a small portfolio rather than a single presentation

Strictly improving local moves can miss a path through a temporarily larger but algebraically more useful expression. A bounded Pareto frontier over depth, node count, coefficient size, and field degree could retain a few such states. Only the final published representation would need to beat the original cost. Every state would still need exact value certification, and the frontier would need its own explicit size and work limits.

This is an opportunity for better search, not a reason to relax equality checks. The same principle applies to fourth-root and root-of-unity presentations: the allowed output grammar should be an explicit policy, especially when a user wants real radicals only. The current grammar allows principal complex rational powers and therefore measures a different search problem from real-only square-root denesting.

13 Conclusion

The second corrected implementation has the right basic architecture for a practical, incomplete denester: it treats discovery as fallible and places a verification gate between a proposal and a replacement. The most valuable next step is to apply that discipline consistently to the surrounding software contracts.

The root recognizer must establish its claimed domain. Every search stage must preserve the incumbent it receives. An index-reduced expression must be offered even when recursion does nothing. A negative cache entry must not outlive the resource conditions that created it. Limits must be checked where proposals and expensive operations are actually generated, and reports must not perform unbounded work after the budget has expired.

The accompanying revision implements those repairs while retaining the existing search machinery. It also adds two mathematically transparent extensions: bounded character reconstruction for small multiquadratic fields and an odd-index trace-norm kernel in quadratic fields. Exact proofs, concrete witnesses, reproducible source generation, independently executed checks, and an explicitly unexecuted native suite make the proposal useful without overstating its validation status.

A Archive contents and rebuild procedure

File	Purpose
<code>article.tex</code> , <code>article.pdf</code>	This self-contained article and its rendered edition.

File	Purpose
StradFixed3.wl	Standalone proposed package in a separate public context.
StradFixed2.original.wl	Exact reviewed baseline, retained for reproducibility.
build_revision.py	Hash-checking, anchor-checked source generator.
revision_definitions.wl	Replacement private definitions and new kernels.
StradFixed2-to-3.diff	Human-readable unified source comparison.
StradFixed3Tests.wlt, run_tests.wls	Native Wolfram tests and runner; not executed in this review.
independent_checks.py, independent_results.json	Executed exact mathematical/source checks and their full machine-readable results.
README.md, source-manifest.json	Usage, qualification notes, source provenance, and relevant paths.
SHA256SUMS.txt, build_pdf.sh	Artifact-integrity hashes and PDF rebuild commands.

To reproduce the proposed source and independent checks:

```
python build_revision.py
python independent_checks.py
```

To rebuild this article, run pdfLaTeX twice from the archive directory. No external images, bibliography database, local font files, or custom class are required. The document uses standard TeX Live/MiKTeX packages and embedded scalable Latin Modern fonts.

B Source locations for the principal findings

All line references in this appendix refer to the archived, hash-verified `StradFixed2.original.wl`, not to the generated revision.

Concern	Lines	Inspection target
Domain admission	143–166	Opaque objects admitted by precision/no-Real test.
Cost accounting	168–181	Subtractive radical count and atom bit-size scans.
Equality status	183–221	Numeric Different branch; exact-oracle fallback.
Acceptance/polishing	224–255	Single gate; size check after candidate entry.
Rationalization	261–279	Polynomial inverse and exact product certificate.
Multisurd ansatz	345–375	Sparse generator sets, FactorInteger, ansatz expansion.
Linear roots/index reduction	420–455	Together call, recursive-progress condition.
Multiplier expansion	560–572	Full discriminant factorization; batch guard placement.
Incumbent resets	598–620	Stage calls that omit the incumbent.

Concern	Lines	Inspection target
Memoization	628–639	Target-only key; unchanged-result storage.
Session/report	692–728	Scope of constrained evaluation and report cost scans.
Public dispatch	736–742	OptionsPattern tail versus validation contract.

References

- [1] Vladimir Reshetnikov, RadicalDenest repository, *StradFixed2.wl*, commit 9000d6f533a68ee7659a6830ad25bba401f800a5.
<https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/src/corrected/StradFixed2.wl>. The exact file is included in the archive.
- [2] RadicalDenest repository, *Unified analysis B*,
src/code-review/unified-B/unified_analysis_B.tex, same commit.
<https://github.com/VladimirReshetnikov/RadicalDenest/tree/9000d6f533a68ee7659a6830ad25bba401f800a5/src/code-review/unified-B>.
- [3] RadicalDenest repository, *Radical denesting: unified report*,
docs/report/radical_denesting_unified.tex, same commit.
<https://github.com/VladimirReshetnikov/RadicalDenest/tree/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/report>.
- [4] RadicalDenest repository, *Corrected re-typesettings of six articles*,
docs/literature/RETYPESET_ARTICLES.md, same commit.
https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/literature/RETYPESET_ARTICLES.md.
- [5] David J. Jeffrey and Albert D. Rich, *Simplifying Square Roots of Square Roots by Denesting*, chapter 4. Consulted the repository’s corrected 2026 re-typesetting, especially Sections 4.4–4.5, at <https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/literature/papers/jeffrey-Simplifying-square-roots-of-square-roots-by-denesting/retypeset-2026/denest.tex>.
- [6] Allan Borodin, Ronald Fagin, John E. Hopcroft, and Martin Tompa, *Decreasing the Nesting Depth of Expressions Involving Square Roots* (1985). Consulted the repository’s corrected 2026 re-typesetting, including its explicit editorial qualifications:
<https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/literature/papers/bfht-Decreasing-the-nesting-depth-of-expressions-involving-square-root/retypeset-2026/symb85.tex>.
- [7] Susan Landau, *Simplification of Nested Radicals*, 1989 extended abstract. Consulted the corrected 2026 repository re-typesetting and its editorial distinction from the final 1992 SIAM Journal on Computing paper, DOI 10.1137/0221009:
<https://github.com/VladimirReshetnikov/RadicalDenest/blob/9000d6f533a68ee7659a6830ad25bba401f800a5/docs/literature/papers/landau92-Simplification-of-nested-radicals/retypeset-2026/landau1989.tex>.
- [8] Wolfram Research, *Root*, Wolfram Language documentation, consulted September 2026.
<https://reference.wolfram.com/language/ref/Root.html>.

- [9] Wolfram Research, *PossibleZeroQ*, particularly the Method -> "ExactAlgebraics" contract, consulted September 2026.
<https://reference.wolfram.com/language/ref/PossibleZeroQ.html>.
- [10] Wolfram Research, *RootReduce*, consulted September 2026.
<https://reference.wolfram.com/language/ref/RootReduce.html>.
- [11] Wolfram Research, *TimeConstrained*, details on CPU accounting, interrupt granularity, and abort protection, consulted September 2026.
<https://reference.wolfram.com/language/ref/TimeConstrained.html>.
- [12] Wolfram Research, *MemoryConstrained*, consulted September 2026.
<https://reference.wolfram.com/language/ref/MemoryConstrained.html>.
- [13] Wolfram Research, *ToRadicals*, consulted September 2026.
<https://reference.wolfram.com/language/ref/ToRadicals.html>.